Streaming API v8

Installation

Licensing

This software uses libraries and code of FFmpeg ↗ licensed under the LGPLv2.1 ↗ and its source is included in the distributed Docker image as `FFmpeg.tar.gz`.

Docker

1. Follow the steps here ↗ to install Docker Engine on your host machine.

2. Verify your installation by running `docker run hello-world`.

TensorRT (GPU)

Install GPU driver

⚠ This setup guide is for Linux-based machines. For Windows machines, we recommend following the official guide from Docker ↗. You need to use Windows 10, version 1903 (or higher) or Windows 11, with the *most recent Linux kernel in WSL2* (it can be updated with `wsl --update`).

1. Download and run the appropriate NVIDIA driver installer ↗. To use our latest containers, make sure to install a driver compatible with TensorRT 8.6.1. The recommended version is CUDA Toolkit 12.0+.

2. Verify your driver installation by running `nvidia-smi`. The output should start with the following:

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 525.125.05   Driver Version: 529.08       CUDA Version: 12.0     |
|-------------------------------+----------------------+----------------------+
```

Install NVIDIA Container Runtime

1. Install gcc and make

```
sudo apt-get update
sudo apt-get install -y gcc make
```

2. Install NVIDIA Container Runtime

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID) \
    && curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey |
sudo apt-key add - \
    && curl -s -L https://nvidia.github.io/nvidia-
docker/$distribution/nvidia-docker.list | sudo tee
/etc/apt/sources.list.d/nvidia-docker.list
sudo apt-get update
sudo apt-get install nvidia-container-toolkit
```

3. Restart Docker

```
sudo systemctl restart docker
```

4. Verify by running the command:

```
docker run -it --rm --gpus=all ubuntu nvidia-smi
```

You should see the version information similar to the following or greater:

```
+-----------------------------------------------------------------
----------+
| NVIDIA-SMI 525.125.05   Driver Version: 529.08       CUDA Version:
12.0     |
|-------------------------------+----------------------+------------
----------+
```

Streaming Container

Ensure you have Docker Engine installed and that you're logged in with the Docker user we've given access to in our repository. Then, run the following to download the image from the Paravision repository:

```
docker pull containers.paravision.ai/streaming/streaming:<tag>
```

Ports

Expose ports 5000, 5050 and 5051 on your machine. 5000 is used to make requests to start and stop decoding, 5050 and 5051 are used to receive face events and important system alerts respectively, both via Socket.IO ↗.

Run Container

Run the following to start the streaming app with appropriate port mapping.

GPU

```
docker run --rm --gpus=all -d --name paravision-streaming -p
5000:5000 -p 5050:5050 -p 5051:5051 --shm-size=3g
containers.paravision.ai/streaming/streaming:<tag>
```

CPU

```
docker run --rm -d --name paravision-streaming -p 5000:5000 -p
5050:5050 -p 5051:5051 --shm-size=3g
containers.paravision.ai/streaming/streaming:<tag>
```

Note: The docker container runs as root by default. To run the container as a non-root user, please add the flag `-u <user id>:<group id>` in your run command. For example, to run the container as your current user, you can use `-u $(id -u ${USER}):$(id -g ${USER})`. TensorRT engine files are stored in the container's `/tmp` directory so that non-root users can access them.

Options

The following options can be configured via environment variables when running the container. For example, `-e PV_INFERENCE_WORKERS=4,0,2,3`.

PV_FACE_ENABLED

Enables/disables the loading of the face processing SDKs and components of the Streaming container at startup. When disabled, face detection and tracking will not be possible by the container. This reduces memory consumption by the container if face processing is not needed.

Default: `True`

> This will be automatically enabled if all other processing components are disabled or if PV_ATTRIBUTES_ENABLED is enabled.

> This behavior is permanent after starting up the container and the container will have to be restarted for a change to this environmental variable to take effect. The status of this component being enabled or not can be acquired from the Application info API.

PV_ATTRIBUTES_ENABLED

Enables/disables the loading of the attributes processing SDKs and components of the Streaming container at startup. When disabled, attributes detection will not be possible by the container. This reduces memory consumption by the container if attributes processing is not needed.

Default: `True`

> This behavior is permanent after starting up the container and the container will have to be restarted for a change to this environmental variable to take effect. The status of this component being enabled or not can be acquired from the Application info API.

PV_PERSON_ENABLED

Enables/disables the loading of the person processing SDKs and components of the Streaming container at startup. When disabled, person detection and tracking will not be possible by the container. This reduces memory consumption by the container if person processing is not needed.

Default: `True`

> ⚠ This behavior is permanent after starting up the container and the container will have to be restarted for a change to this environmental variable to take effect. The status of this component being enabled or not can be acquired from the Application info API.

PV_VEHICLE_ENABLED

Enables/disables the loading of the vehicle processing SDKs and components of the Streaming container at startup. When disabled, vehicle detection and tracking will not be possible by the container. This reduces memory consumption by the container if vehicle processing is not needed.

Default: `True`

> ⚠ This behavior is permanent after starting up the container and the container will have to be restarted for a change to this environmental variable to take effect. The status of this component being enabled or not can be acquired from the Application info API.

PV_DETECT_FACES

Sets the default behavior for processing face detections with the `start_decode` API, but can be overwritten by the corresponding API parameter.

Default: `PV_FACE_ENABLED`

PV_DETECT_PERSONS

Enable persons detection by default, can be overwritten per stream in `start_decode` parameters.

Default: `PV_PERSON_ENABLED`

PV_DETECT_VEHICLES

Enable vehicles detection by default, can be overwritten per stream in `start_decode` parameters.

Default: `PV_VEHICLE_ENABLED`

PV_INFERENCE_WORKERS

The number of workers processing frames from a stream. Increase the default if you require a higher throughput of processing frames. It supports comma separated integer value for multiple GPU support: each integer must be between 0 to 6. Index of the integer matches directly with the GPU indexes found on `nvidia-smi`.

Default: `1`

PV_POSTPROCESSOR_WORKERS

The number of workers processing faces from frames. Increase the default if you require a higher throughput of processing frames. It supports comma separated integer value for multiple GPU support: each integer must be between 0 to 6. Index of the integer matches directly with the GPU indexes found on `nvidia-smi`.

Default: `1`

PV_DECODING_DEVICES

Indexes of devices to be used for stream decoding (FFmpeg process). To use multiple GPUs, you can provide a list of comma separated indexes (indexes of the

GPUs can be found in `nvidia-smi`), i.e. `PV_DECODING_DEVICES=0,1`. In CPU mode, the CPU is always represented as a single device with index 0.

Default: `0`

PV_MAX_FRAMES_IN_QUEUE

The maximum number of frames that the decoder queue will hold. Frames will be dropped once the queue size reaches this number. Tweak this depending on the type of stream and memory available on the host system. For example, a 500 frame limit for a 1080p stream corresponds to a memory cap of about 3GB. You can raise/lower the cap by increasing/decreasing this variable.

Default: `500`

PV_READ_FRAME_MAX_RETRIES

The number of retries when reading frames for a given stream.

Default: `2`

PV_RETRY_WAIT_PERIOD

The number of seconds to wait between attempts to decode a stream after a read frame error.

Default: `60`

PV_READ_TIMEOUT_SECONDS

The number of seconds to wait when reading frame for a given stream.

Default: `10`

PV_TRT_ENGINE_PATH

Path under which compiled models will be stored. In GPU mode the SDK models are serialized based on the hardware during startup of the application. Specifying this path allows caching the serialized model files, allowing for faster subsequent startups on the same host. Best paired with `-v HOST_DIR:PV_TRT_ENGINE_PATH` volume specification in docker command line to keep the output in persistent directory on the host.

Default: `/tmp`

> ⚠ The compiled models don't automatically update on changes. Make sure to clear out the cache directory when switching between streaming container versions.

PV_OMIT_EMPTY_FRAMES

If set to `True`, no Socket.IO events will be sent for frames with no entities (faces, persons or vehicles) detected. Default:

`False`

PV_FACE_ACCEPTABILITY_TRACKING_THRESHOLD

Faces with acceptability ↗ lower than this value will never receive a tracking ID.

Default: `0.15`

PV_FACE_QUALITY_TRACKING_THRESHOLD

Faces with quality ↗ lower than this value will never receive a tracking ID.

Default: `0.25`

PV_FACE_SIMILARITY_TRACKING_THRESHOLD

Minimum similarity ↗ to the previous tracked face object for a face to have assigned the same tracking ID.

Default: `2.8`

PV_FACE_TRACKING_KEEP_HIGHEST_QUALITY

When enabled, the embedding for the highest quality face detected for a tracked face will be stored and used as a comparison against unknown faces detected. When disabled, the tracked face's embedding will be updated on every new encounter of that face.

Default: `False`

PV_TRACKING_QUALITY_THRESHOLD_HISTORY_SIZE

Utilized when the face-quality tracking threshold is set to `-1`.
The quantity of the most recent quality values to store for the faces being tracked. These quality values are utilized in conjunction with the tracking-quality percentile threshold to auto-adjust the face quality threshold utilized for tracking.

Default: `100`

PV_TRACKING_QUALITY_PERCENTILE_THRESHOLD

Utilized when face-quality tracking threshold is set to `-1`.
The quality value surpassing this percentile threshold from the history of face qualities being stored for the tracked faces will then be utilized for the subsequent face quality threshold.

Default: `20`

PV_VEHICLE_TRACKING_DURATION_SECONDS

Time in seconds until a tracking ID for a vehicle expires after its last detection

Default: `3.0`

PV_VEHICLE_TRACKING_MIN_DETECTIONS_PER_TRACK

Minimum number of detection of the same vehicle needed before it is assigned a tracking ID.

Default: `10`

PV_VEHICLE_TRACKING_VELOCITY_THRESHOLD

Minimum velocity (pixels per frame) threshold for a tracked vehicle to be considered moving.

Default: `15.0`

PV_VEHICLE_TRACKING_DETECTION_THRESHOLD

Minimum detection quality threshold for a detected vehicle to be considered for tracking.

Default: `0.25`

PV_PERSON_TRACKING_DURATION_SECONDS

Time in seconds until a tracking ID for a person expires after its last detection.

Default: `3.0`

PV_PERSON_TRACKING_MIN_DETECTIONS_PER_TRACK

Minimum number of detection of the same person needed before it is assigned a tracking ID.

Default: `10`

PV_PERSON_TRACKING_VELOCITY_THRESHOLD

Minimum velocity (pixels per frame) threshold for a tracked person to be considered moving.

Default: `2.0`

PV_PERSON_TRACKING_DETECTION_THRESHOLD

Minimum detection quality threshold for a detected person to be considered for tracking.

Default: `0.45`

PV_OMIT_UNTRACKED_OBJECTS

Setting this flag to `True` will filter out objects that are not tracked from Socket.IO events.

Default: `False`

PV_OUTPUT_FRAMES_ENABLED

Enable outputting stream frames in Frame events by default, can be overwritten per stream in `start_decode` parameters. Events will include additional `frame` field that is a stream frame on which results are based (faces, etc.). The frame image is provided as base64 encoded string.

Default: `False`

> Enabling this option will have negative impact on the performance of the Streaming container.

PV_OUTPUT_ATTRIBUTES_IMAGES

Controls whether or not to output the `attributes_images` for each face detected in the Streaming Socket.IO Outputs by default, can be overwritten per stream using

the `output_attributes_images` in `start_decode` parameters. The `attributes_images` will be populated in the `images` field for each face object. The value for this field is an array containing 2 base64-encoded images which can be directly provided into the Paravision Processor API's `process_prepared_face_image` input to process the Attributes for the corresponding face.

Default: `False`

> Enabling this option will have negative impact on the performance of the Streaming container.

PV_OUTPUT_FRAMES_SCALE

If outputting frames is enabled, frames will be scaled using this coefficient. The coefficient affects performance of the container. For decreasing performance loss, it's advised to decrease the coefficient (within the 0.0-1.0 range).

Default: `1.0`

Both the image width and height are multiplied by the coefficient - for example when it's `0.5` then the image will be half the width and half the height, thus have 4 times fewer pixels.

PV_PROMETHEUS_METRICS

When set to `on`, enables Prometheus ↗ metrics in the Streaming container.

Metrics will be exposed on the port `8092`.

Available Counters ↗:

Name	Labels	Description
`pv_streaming_output_faces_total`	`stream_name`	Total count of faces output by stream.

`pv_streaming_output_persons_total`	`stream_name`	Total count of persons output by stream.
`pv_streaming_output_vehicles_total`	`stream_name`	Total count of vehicles output by stream.
`pv_streaming_worker_frames_total`	`type`, `worker_id`, `stream_name`	Total count of frames processed by every worker in the system.
pv_streaming_alerts_total	type, stream_name	Total count of alerts output by Streaming API

Available Gauges ↗:

Name	Labels	Description
`pv_streaming_tracked_faces`	`stream_name`	Total count of faces that is currently being tracked for stream.
`pv_streaming_queue_size`	`type`, `worker_id`	Sizes of different `multiprocessing.Queue`s across the system.
`pv_streaming_shared_memory_available_gigabytes`	None	Shared memory available in the container (in gigabytes).
`pv_instance_gpu_usage_ratio`	`GPU_id`	GPU utilization percentage (GPU mode only).
`pv_instance_gpu_memory_usage_ratio`	`GPU_id`	Percentage of GPU memory usage (GPU mode only).
`pv_instance_cpu_usage_ratio`	None	CPU utilization percentage (CPU mode only).
`pv_instance_memory_usage_ratio`	None	Percentage of RAM memory usage (CPU mode only).
`pv_streaming_tracked_persons`	`stream_name`	Persons currently tracked in the stream
`pv_streaming_tracked_vehicles`	`stream_name`	Vehicles currently tracked in the stream

Available Histograms ↗:

Name	Labels	Description
`pv_streaming_frame_processing_time_seconds`	`stream_name`	Time taken to process each frame (includes *all* steps). Values higher than 10 seconds are clipped.

PV_SOURCE_TIMESTAMP

When enabled by setting this environmental variable to `ON`, Streaming will attempt to find the FFmpeg PRFT ↗ frame metadata field provided within each frame of the client's video stream and output this timestamp (if found) in the Streaming Socket.IO Outputs via `source_timestamp` with the value as an ISO formatted timestamp `string`.

> Streaming will always output the `source_timestamp` from its events, however if the environmental variable is not provided OR if the PRFT frame metadata is not found in the frame processed, this field will contain a `null` value.

Clients

Streaming provides HTTP API with JSON payloads for control, and Socket.IO **protocol v5** with binary payloads for receiving events.

Make sure libraries you want to use support the above requirements. See recommended client libraries below and samples for more detailed instructions.

> When evaluating Socket.IO client libraries make sure they support Socket.IO **protocol** version **v5** - it is supported by official Socket.IO server 3.x/4.x which is more often referenced in libraries documentations.

C#

- C# has built-in support for HTTP and JSON
- Semi official SocketIOClient ↗ (`>=3.0.3`) for Socket.IO

C++

- cpp-httplib ↗ (`>=v0.9.7`) for HTTP
- Official Socket.IO C++ Client ↗ (`>=3.1.0`) for Socket.IO and bundled with it RapidJSON ↗ for JSON

> Current version has a bug with regards to parsing binary messages. It needs to be patched with patch from this report ↗.

Java

- OkHttp ↗ (`>=4.9.1`) for HTTP
- Java has built-in support for JSON
- Official Socket.IO-client Java ↗ (`>=2.0.1`) for Socket.IO

Node

- axios ↗ (`>=0.24`) for HTTP
- Javascript has built-in JSON support
- Official socket.io-client ↗ (`>=4.3`) for Socket.IO

Python

- Requests ↗ (`>=2.26.0`) for HTTP
- Python has built-in JSON support
- Semi official python-socketio ↗ (`>=5.4.1`) for Socket.IO

> Make sure to install client library with `python-socketio[client]` ↗.

Recommendations

Workers Count

Maximum throughput highly depends on number of workers (both `PV_INFERENCE_WORKERS` and `PV_POSTPROCESSOR_WORKERS`).

- Inference workers process frames from the input streams and output faces from them.
- Postprocessor workers take the faces and run face recognition and matching on them to ensure the stability of the `tracking_ids` (if enabled for specific stream) as well as run some additional postprocessing steps to prepare the final output payloads.

Depending on the number of frames the whole system needs to process it might be needed to increase the number of `PV_INFERENCE_WORKERS` (to be able to efficiently extract faces from them), and then depending on the potential number of faces in those streams increase the `PV_POSTPROCESSOR_WORKERS`.

Decoding Devices

Every stream requires a separate decoding process. By default, all these processes utilize only the first available GPU device. This might cause workers started on this device to work less effectively or limit the maximum number of workers that can be started on this device. To prevent this, the decoding processes can be spanned across multiple GPUs using `PV_DECODING_DEVICES`.

Tracking Thresholds

All `*_TRACKING_*_THRESHOLD` parameters can be tuned to achieve optimal tracking performance. In general, too low thresholds result in erroneous tracking assignments. On the other hand, too high thresholds may result in losses in tracking due to ignoring faces on intermediate frames.

Increasing the thresholds reduces the required workload and can increase system throughput.

Shared Memory

Shared memory is used internally for frames and face images storage. Depending on number of streams, their resolution, PV_MAX_FRAMES_IN_QUEUE, PV_INFERENCE_WORKERS, and the decoding `frame_rate` value from Request Parameters, it might be necessary to tune its size. For example 1080p frame weighs about 6MB so for up to 500 frames in the queue (assuming one Inference worker) you might need up to 3GB of shared memory. However, if the setup is tuned properly (not overloaded) you are very unlikely to reach that value, as when the throughput allows it queues should not be able to reach this size.

You can do that with `--shm-size` docker ↗ flag, that we by default set to `3g` in the commands above.

The format is `<number><unit>`. `number` must be greater than `0`. Unit is optional and can be `b` (bytes), `k` (kilobytes), `m` (megabytes), or `g` (gigabytes). If you omit the unit, the system uses bytes. If you omit the size entirely, the system uses `64m`.

> On shared memory overload the application will drop frames acquired while the shared memory capacity is reached. The application will also send out error alerts (see Receiving Alerts)and emit error logs to alert the user when this occurs. It is recommended to immediately take one of the following actions when this occurs:
>
> - Stop decoding/processing streams until this error subsides (see Stop Decoding)
> - Lower the `frame_rate` value for the streams processed (see Request Parameters)
> - Increase the `shm-size`

Tracking Configurations

Person

The recommended base tracking configurations for Person tracking with the decoding frame rate (`frame_rate`) set to `10`:

- velocity threshold: `2.0`
- detection confidence threshold: `0.45`
- min detection per track: `2`
- max track duration seconds: `1.0`

Vehicle

The recommended base tracking configurations for Vehicle tracking with the decoding frame rate (`frame_rate`) set to `10`:

- velocity threshold: `10.0`
- detection confidence threshold: `0.25`
- min detection per track: `3`
- max track duration seconds: `2.33`

Usage

The Paravision Streaming app allows you to receive face, person and vehicle data for frames in an IP camera stream.

> The following stream codecs are supported:
>
> - for CPU: all supported by FFmpeg ↗,
> - for GPU: `av1`, `h264`, `hevc`, `mjpeg`, `mpeg1`, `mpeg2`, `mpeg4`, `vc1`, `vp8`, `vp9`

> This is not a RESTful API. The HTTP endpoints let you handle stream decoding and receive events via sockets.

Control Commands

Control endpoint (HTTP) listens on port `5000`. It allows to start, stop and get information about the available streams. It also exposes simple health check for the application.

Unless otherwise stated, all arguments for POST requests need to be passed as JSON objects (`application/json`).

Data is returned in response body as JSON-formatted objects (`application/json`).

Start Decoding

```
POST /start_decode
```

To start decoding a stream, make a `start_decode` request with `source` and `name` in the JSON request body. The `source` value is the url where the video stream is

pulled from. We support streams from any modern IP camera. The `name` value is a unique identifier for a stream. An error will be thrown if attempting to start a stream with a `name` that's already being decoded.

To receive data, see Receiving Frame Events.

Request Parameters

The parameters are optional, unless otherwise stated.

General

Parameter	Description
name (*string*, ***required***)	A unique identifier for the stream.
source (*string*, ***required***)	The source URL for the stream.
frame_rate (*float*, default: 1.0)	The rate at which to sample the frames in the stream for processing. When a value is given that is above the stream's native frame rate, the frames will be duplicated to achieve the provided frame rate. When a value is given that is below the stream's native frame rate, frames will be dropped to reach the given frame rate. Values between `0.0` and `1.0` are accepted and the frames will be sampled less than once a second to achieve the given frame rate value.
omit_empty_frames (*bool*)	Override for `PV_OMIT_EMTPY_FRAMES` value.
probe_timeout (*float*, default: 20)	The stream format detection timeout in seconds.
rotation (*int*, default: 0)	The number of degrees to rotate the input stream counter-clockwise. Must be one of 0, 90, 180, 270.

skip_identical_frames (*bool*, default: false)	If true, frames will be skipped if no change is detected since the previous frame.
omit_untracked_objects (*bool*, default: false)	Override for `PV_OMIT_UNTRACKED_OBJECTS` value.
output_frames_enabled (*bool*, default: false)	Whether or not to send frame images in Frame events. Override for `PV_OUTPUT_FRAMES_ENABLED` value.
output_frames_scale (*float*, default: 1.0)	The scale at which frame images should be sent in Frame events. Override for `PV_OUTPUT_FRAMES_SCALE` value.

Face

Parameter	Description
detect_faces (*bool*, default: PV_DETECT_FACES)	Whether or not to process for face detection. Override for `PV_DETECT_FACES` value. If enabled but PV_FACE_ENABLED is disabled, a `400` error will result from the API response with the message "Face processing is not enabled".
detect_mask (*bool*, default: PV_ATTRIBUTES_ENABLED)	Whether or not to process a `mask` confidence value for each face detected. If enabled but PV_ATTRIBUTES_ENABLED is disabled, a `400` error will result from the API response with the message "Attributes processing is not enabled".
track_faces (*bool*, default: PV_DETECT_FACES)	Whether or not to track faces in the stream. If enabled but PV_FACE_ENABLED is disabled, a `400` error will result from the API response with the message "Face processing is not enabled".

face_acceptability_tracking_threshold (*float*)	Override for `PV_FACE_ACCEPTABILITY_TRACKING_THRESHOLD` value.
face_expanded_image_scale (*float*, default: 1.5)	The scale at which to expand the recognition input face image.
face_quality_tracking_threshold (*float*)	Override for `PV_FACE_QUALITY_TRACKING_THRESHOLD` value.
face_similarity_tracking_threshold (*float*)	Override for `PV_FACE_SIMILARITY_TRACKING_THRESHOLD` value.
face_tracking_max_frames_per_track (*int*, default: -1)	The maximum number of frames a face will be persisted after its first detection until it is dropped (whether or not it is detected or tracked in subsequent frames). Set to `-1` to disable.
face_tracking_min_detections_per_track (*int*, default: 0)	The minimum number of consecutive face detections that are needed for the same face before it's considered tracked.
face_tracking_duration_seconds (*float*, default: 20.0)	The length of time in seconds until a tracking ID for a face expires after its last detection. Set to `-1` to disable.
tracking_min_face_size (*float*, default: 0)	Minimum height **and** width of a tracked face's bounding box. Faces smaller in either dimension will not receive a tracking ID. Set to 0 to effectively disable.
tracking_quality_threshold_history_size (*int*, 100)	Utilized when face-quality tracking threshold is set to `-1`. The quantity of the most recent quality values to store for the faces being tracked. These quality values are utilized in conjunction with the tracking-quality percentile threshold to auto-adjust the face quality threshold utilized for tracking.
	Utilized when face-quality tracking threshold is set to `-1`. The quality value surpassing this percentile threshold from

tracking_quality_percentile_threshold (*int*, default: 20)	the history of face qualities being stored for the tracked faces will then be utilized for the subsequent face quality threshold.
tracking_group (*string*)	When set, tracking group to utilize for the faces tracked in this stream. When more than 1 stream processed contains the same `tracking_group`, the faces tracked in each stream will be grouped together and utilize the same tracking IDs for similar faces found in each.
output_attributes_images (bool, default: false)	Whether or not to send the `attributes_images` in the output Frame events. When enabled, these images will be sent out for each face detected. Override for PV_OUTPUT_ATTRIBUTES_IMAGES value.

Person

Parameter	Description
detect_persons (*bool*, default: PV_DETECT_PERSONS)	Whether or not to process for person detection. Override for `PV_DETECT_PERSONS` value. If enabled but PV_PERSON_ENABLED is disabled, a `400` error will result from the API response with the message "Person processing is not enabled".
track_persons (*bool*, default: PV_DETECT_PERSONS)	Whether or not to track persons in the stream. If enabled but PV_PERSON_ENABLED is disabled, a `400` error will result from the API response with the message "Person processing is not enabled".
person_expanded_image_scale (*float*, default: 1.0)	The scale at which to expand the person image.

person_tracking_duration_seconds (*float*, default: 1.0)	Override for `PV_PERSON_TRACKING_DURATION_SECONDS` value.
person_tracking_min_detections_per_track (*int*, default: 1)	Override for `PV_PERSON_TRACKING_MIN_DETECTIONS_PER_TRACK` value.
person_tracking_velocity_threshold (*float*, default: 2.0)	Override for `PV_PERSON_TRACKING_VELOCITY_THRESHOLD` value.
person_tracking_detection_threshold (*float*, default: 0.45)	Override for `PV_PERSON_TRACKING_DETECTION_THRESHOLD` value.

Vehicle

Parameter	Description
detect_vehicles (*bool*, default: PV_DETECT_VEHICLES)	Whether or not to process for vehicle detection. Override for `PV_DETECT_VEHICLES` value. If enabled but PV_VEHICLE_ENABLED is disabled, a `400` error will result from the API response with the message "Vehicle processing is not enabled".
track_vehicles (*bool*, default: PV_DETECT_VEHICLES)	Whether or not to track vehicles in the stream. If enabled but PV_VEHICLE_ENABLED is disabled, a `400` error will result from the API response with the message "Vehicle processing is not enabled".
vehicle_expanded_image_scale (*float*, default: 1.0)	The scale at which to expand the vehicle image.
vehicle_tracking_duration_seconds (*float*, default: 2.33)	Override for `PV_VEHICLE_TRACKING_DURATION_SECONDS` value.

vehicle_tracking_min_detections_per_track (*int*, default: 3)	Override for `PV_VEHICLE_TRACKING_MIN_DETECTIONS_PER_TRACK` value.
vehicle_tracking_velocity_threshold (*float*, default: 15.0)	Override for `PV_VEHICLE_TRACKING_VELOCITY_THRESHOLD` value.
vehicle_tracking_detection_threshold (*float*, default: 0.25)	Override for `PV_VEHICLE_TRACKING_DETECTION_THRESHOLD` value.

Note: Remember to percent-encode `source` URL's parts if needed.

Example: Stream located at `rtsp://<username>:<password>@camera1/stream` expects authentication using `adm!n` and `pas#word` credentials. They should be encoded into `adm%21n` and `pas%23word`, respectively, and the URL will be `rtsp://adm%21n:pas%23word@camera1/stream`.

For the list of characters that must be percent-encoded, see: Percent-encoding on Wikipedia ↗ or RFC 3986 ↗.

Example

Shell

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{ "name": "stream_name", "source": "stream_url",
"detect_mask": true }' \
  http://localhost:5000/start_decode
```

The above command returns JSON structured like this:

```
{
    "message": "Stream decoding successfully started",
    "name": "stream_name",
    "source": "stream_url",
    "success": true
}
```

Stop Decoding

```
POST /stop_decode
```

To stop decoding a stream, make a `stop_decode` request with the stream `name` in the JSON request body. An error will be thrown if the stream with `name` was not being decoded.

Request Parameters

Parameter	Description
name (*string*, ***required***)	A unique identifier for the stream.

Example

Shell

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{ "name": "stream_name" }' \
  http://localhost:5000/stop_decode
```

The above command returns JSON structured like this:

```
{
    "message": "Stream decoding successfully terminated",
    "success": true
}
```

Get Streams

```
GET /streams
```

The get streams endpoint will return a list of currently decoding streams.

Example

Shell

```
curl http://localhost:5000/streams
```

The above command returns JSON structured like this:

```
{
    "message": "Successfully retrieved decoding streams",
    "streams": [
        {
            "detect_faces": true,
            "detect_mask": true,
            "detect_persons": true,
            "detect_vehicles": true,
            "face_acceptability_tracking_threshold": 0.15,
            "face_expanded_image_scale": 1.5,
            "face_quality_tracking_threshold": 0.25,
            "face_similarity_tracking_threshold": 2.8,
            "face_tracking_duration_seconds": 20.0,
            "face_tracking_max_frames_per_track": -1,
            "face_tracking_min_detections_per_track": 0,
            "frame_rate": 1.0,
            "inference_worker_id": 0,
            "metadata": {
                "codec": "mpeg4",
                "frame_rate": "50/1",
                "height": 1080,
                "width": 1920
            },
            "name": "stream_name1",
            "omit_empty_frames": false,
            "omit_untracked_objects": false,
            "output_attributes_images": true,
            "output_frames_enabled": false,
            "output_frames_scale": 1.0,
            "person_expanded_image_scale": 1.0,
            "person_tracking_detection_threshold": 0.45,
            "person_tracking_duration_seconds": 1.0,
            "person_tracking_min_detections_per_track": 1,
            "person_tracking_velocity_threshold": 2.0,
            "postprocessor_worker_id": 0,
            "rotation": 0,
            "skip_identical_frames": false,
            "source": "stream_url1",
            "track_faces": true,
            "track_persons": true,
            "track_vehicles": true,
            "tracking_group": "",
            "tracking_min_face_size": 0,
            "tracking_quality_percentile_threshold": 20,
            "tracking_quality_threshold_history_size": 100,
            "vehicle_expanded_image_scale": 1.0,
            "vehicle_tracking_detection_threshold": 0.25,
            "vehicle_tracking_duration_seconds": 2.33,
```

```
            "vehicle_tracking_min_detections_per_track": 3,
            "vehicle_tracking_velocity_threshold": 15.0
        },
        {
            "detect_faces": true,
            "detect_mask": true,
            "detect_persons": true,
            "detect_vehicles": true,
            "face_acceptability_tracking_threshold": 0.15,
            "face_expanded_image_scale": 1.5,
            "face_quality_tracking_threshold": 0.25,
            "face_tracking_duration_seconds": 2.0,
            "face_tracking_max_frames_per_track": -1,
            "face_tracking_min_detections_per_track": 0,
            "frame_rate": 1.0,
            "inference_worker_id": 0,
            "metadata": {
                "codec": "mpeg4",
                "frame_rate": "2997/100",
                "height": 1080,
                "width": 1920
            "omit_empty_frames": false,
            "omit_untracked_objects": false,
            "output_attributes_images": false,
            "output_frames_enabled": false,
            "output_frames_scale": 1.0,
            "person_expanded_image_scale": 1.0,
            "person_tracking_detection_threshold": 0.45,
            "person_tracking_duration_seconds": 1.0,
            "person_tracking_min_detections_per_track": 1,
            "person_tracking_velocity_threshold": 2.0,
            "postprocessor_worker_id": 0,
            "rotation": 0,
            "skip_identical_frames": false,
            "source": "stream_url2",
            "track_faces": true,
            "track_persons": true,
            "track_vehicles": true,
            "tracking_group": "",
            "tracking_min_face_size": 0,
            "tracking_quality_percentile_threshold": 20,
            "tracking_quality_threshold_history_size": 100,
            "vehicle_expanded_image_scale": 1.0,
            "vehicle_tracking_detection_threshold": 0.25,
            "vehicle_tracking_duration_seconds": 2.33,
```


Get Stream

```
GET /streams/<stream_name>
```

Example

Shell

```
curl http://localhost:5000/streams/<stream_name>
```

```
{
    "message": "Successfully retrieved decoding stream",
    "stream": {
        "detect_faces": true,
        "detect_mask": true,
        "detect_persons": true,
        "detect_vehicles": true,
        "face_acceptability_tracking_threshold": 0.15,
        "face_expanded_image_scale": 1.5,
        "face_quality_tracking_threshold": 0.25,
        "face_similarity_tracking_threshold": 2.8,
        "face_tracking_duration_seconds": 20.0,
        "face_tracking_max_frames_per_track": -1,
        "face_tracking_min_detections_per_track": 0,
        "frame_rate": 1.0,
        "inference_worker_id": 0,
        "metadata": {
            "codec": "mpeg4",
            "frame_rate": "50/1",
            "height": 1080,
            "width": 1920
        },
        "name": "stream_name1",
        "omit_empty_frames": false,
        "omit_untracked_objects": false,
        "output_attributes_images": false,
        "output_frames_enabled": false,
        "output_frames_scale": 1.0,
        "person_expanded_image_scale": 1.0,
        "person_tracking_detection_threshold": 0.45,
        "person_tracking_duration_seconds": 1.0,
        "person_tracking_min_detections_per_track": 1,
        "person_tracking_velocity_threshold": 2.0,
        "postprocessor_worker_id": 0,
        "rotation": 0,
        "skip_identical_frames": false,
        "source": "stream_url1",
        "track_faces": true,
        "track_persons": true,
        "track_vehicles": true,
        "tracking_group": "",
        "tracking_min_face_size": 0,
        "tracking_quality_percentile_threshold": 20,
        "tracking_quality_threshold_history_size": 100,
        "vehicle_expanded_image_scale": 1.0,
        "vehicle_tracking_detection_threshold": 0.25,
        "vehicle_tracking_duration_seconds": 2.33,
        "vehicle_tracking_min_detections_per_track": 3,
```

```
        "vehicle_tracking_velocity_threshold": 15.0
    },
    "success": true
}
```

Health Check

```
GET /health
```

The health check endpoint will show the health status of the Streaming app. A description of the issues will be returned if they exist.

Example

Shell

```
curl http://localhost:5000/health
```

The above command returns JSON structured like this:

```
{
    "message": "All well!",
    "success": true
}
```

Application info

```
GET /
```

This endpoint will return the application info.

Example

Shell

```
curl http://localhost:5000/
```

The above command returns JSON structured like this:


```
{
    "app_version": "8.0.1",
    "attributes_enabled": true,
    "attributes_model_version": "2.0.0",
    "attributes_sdk_version": "2.0.0",
    "engine": "openvino",
    "face_enabled": true,
    "message": "Successfully retrieved application info",
    "person_enabled": true,
    "person_model_version": "0.4.0",
    "person_sdk_version": "0.5.0",
    "recognition_model_name": "fast",
    "recognition_model_version": "1.0.0",
    "recognition_sdk_version": "9.0.1",
    "success": true,
    "vehicle_enabled": true,
    "vehicle_model_version": "0.4.0",
    "vehicle_sdk_version": "0.5.0"
}
```


Socket.IO Outputs

Outputs - frame events and alerts are broadcasted using Socket.IO ↗ on ports 5050 and 5051 respectively.

Receiving Frame Events

Connection Path	Event
`ws://<host>:5050/<stream_name>`	`frame_decoded`

After decoding of the stream has started, use a Socket.IO library in the platform of your choice to connect to `ws://<host>:5050/<stream_name>` where `stream_name` is the `name` of your decoding stream (also the namespace of the Socket.IO url). Then, listen to face detection events on the `frame_decoded` channel for that url. The value of `stream_name` is the same as the `name` provided to start decoding the stream in `start_decode`.

The events in the response are of a JSON format (sent as Socket.IO binary packets) and includes data for each frame such as bounding boxes, landmarks, acceptability score, yaw, pitch, roll, tracking as well as recognition input and expanded images as base64 encoded strings.

The streaming container can be used directly with Paravision Search Lite to identify tracked faces found in streams against a gallery of embeddings. Alternatively, it can also be used in conjunction with the Processor and Identity or Search API containers. The `recognition_input_image` images from the streaming container can be used to obtain face embeddings using Processor API, which can in turn be used to perform lookups in Identity API to identify faces.

Furthermore, tracking provides a way to determine if the face/identity has already been seen in the last N seconds so lookups don't need to be made repeatedly for the same identity.

Example

C#

```
using System;
using System.Text;
using SocketIOClient;

namespace Example
{
    class Program
    {
        private const string MY_NAMESPACE = "/stream_name";
        private const string STREAM_URL = "ws://<host>:5050";

        static void Main(string[] args)
        {
            var sio = new SocketIO(new Uri(STREAM_URL +
MY_NAMESPACE), new SocketIOOptions
            {
                Transport =
SocketIOClient.Transport.TransportProtocol.WebSocket,
            });

            sio.OnConnected += (sender, ev) =>
            {
                Console.WriteLine("Connected");
            };

            sio.OnDisconnected += (sender, ev) =>
            {
                Console.WriteLine("Disconnected");
            };

            sio.OnError += (sender, error) =>
            {
                Console.WriteLine("Error: {0}", error);
            };

            sio.On("frame_decoded", ev =>
            {

Console.WriteLine(Encoding.UTF8.GetString(ev.GetValue<byte[]>()));
            });

            sio.ConnectAsync().Wait();

            Console.ReadLine();
        }
    }
}
```

C++

```
#include <iostream>
#include <thread>

#include <sio_client.h>

using namespace std;

const char *MY_NAMESPACE = "/stream_name";
const char *STREAM_URL = "ws://<host>:5050";

int main(int argc, char *argv[]) {
  sio::client client;
  client.set_socket_open_listener(
      [&](const string &) { cout << "Connected" << endl; });
  client.set_socket_close_listener(
      [](const string &) { cout << "Disconnected" << endl; });
  client.set_fail_listener(
      []() { cout << "Failure" << endl; });

  client.connect(STREAM_URL);

  client.socket(MY_NAMESPACE)->on_error(
      [&](sio::message::ptr const &msg) { cout << "Error: " << msg-
>get_string() << endl; });

  client.socket(MY_NAMESPACE)->on("frame_decoded",
      [&](const string &, const sio::message::ptr &message, bool,
sio::message::list &) { cout << *message->get_binary() << endl; });

  while (true)
    this_thread::sleep_for(1s);

  return 0;
}
```

Java

```
import io.socket.client.*;
import io.socket.emitter.Emitter;
import io.socket.engineio.client.transports.WebSocket;
import java.net.URISyntaxException;
import java.nio.charset.StandardCharsets;

public class Example {
    public static final String MY_NAMESPACE = "/stream_name";
    public static final String STREAM_URL = "ws://<host>:5050";

    public static void main(String[] args) throws URISyntaxException
{
        IO.Options options =
                IO.Options.builder().setTransports(new String[]
{WebSocket.NAME}).build();
        Socket sio = IO.socket(STREAM_URL + MY_NAMESPACE, options);

        sio.on(
                Socket.EVENT_CONNECT,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        System.out.println("Connected");
                    }
                });

        sio.on(
                Socket.EVENT_DISCONNECT,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        System.out.println("Disconnected");
                    }
                });

        sio.on(
                Socket.EVENT_CONNECT_ERROR,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        String err = (String) args[0];
                        System.out.printf("Error: %s%n", err);
                    }
                });

        sio.on(
                "frame_decoded",
                new Emitter.Listener() {
```

```
                    @Override
                    public void call(Object... args) {
                        String s = new String((byte[]) args[0],
StandardCharsets.UTF_8);
                        System.out.printf(s);
                    }
                });

        sio.connect();

        try {
            while (true) Thread.sleep(Long.MAX_VALUE);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Node

```
const io = require("socket.io-client");

const MY_NAMESPACE = "/stream_name";
const STREAM_URL = "ws://<host>:5050";

var sio = io.connect(STREAM_URL + MY_NAMESPACE, {
  transports: ["websocket"]
});

sio.on("connect", () => {
  console.log("Connected");
})

sio.on("disconnect", () => {
  console.log("Disconnected");
});

sio.on("connect_error", (err) => {
  console.log("Error: " + err);
});

sio.on("frame_decoded", (msg) => {
  var data = JSON.parse(msg);
  console.log(data);
});
```

Python

```
import asyncio
import json

import socketio

MY_NAMESPACE = "/stream_name"
STREAM_URL = "ws://<host>:5050"

sio = socketio.AsyncClient()


@sio.on("connect")
async def connect():
    print("Connected")


@sio.on("disconnect")
async def disconnect():
    print("Disconnected")


@sio.on("connect_error")
async def connect_error(error):
    print("Error: " + error)


@sio.on("frame_decoded", namespace=MY_NAMESPACE)
async def frame_decoded(msg):
    parsed = json.loads(msg.decode())
    print(json.dumps(parsed, indent=2))


async def main(host=STREAM_URL):
    await sio.connect(host, namespaces=[MY_NAMESPACE], transports=
["websocket"])
    await sio.wait()


if __name__ == "__main__":
    asyncio.run(main())
```

The socket events contain JSON structured like this:

```
{
  "faces": [
    {
      "bounding_box": {
        "origin": {
          "x": 376.6867446899414,
          "y": 583.2404399059228
        },
        "width": 92.17248916625977,
        "height": 122.03958330775151
      },
      "landmarks": {
        "left_eye": {
          "x": 1018.018931567669,
          "y": 607.7765703201294
        },
        "right_eye": {
          "x": 1059.093894124031,
          "y": 604.1254320144653
        },
        "nose": {
          "x": 1038.1834517717361,
          "y": 631.0201978683472
        },
        "left_mouth": {
          "x": 1024.7791448831558,
          "y": 655.4442706108093
        },
        "right_mouth": {
          "x": 1058.44459652900,
          "y": 652.093894124031
        }
      },
      "quality": 0.7606140494346619,
      "acceptability": 0.9994677901268005,
      "mask": 0.002386493841186166,
      "tracking_id": "ce3e454c-e171-11e9-9194-0242ac110002",
      "pitch": 2.6782670455016366,
      "yaw": -6.655286879940166,
      "roll": -41.11881119371969,
      "images": {
        "attributes_images": ["Base64 encoded image", "Base64 encoded
image"],
        "recognition_input_image": "Base64 encoded image",
        "expanded": "Base64 encoded image"
      }
    }
  ],
```

```
  "persons": [
    {
      "bounding_box": {
        "origin": {
          "x": 376.6867446899414,
          "y": 583.2404399059228
        },
        "width": 92.17248916625977,
        "height": 122.03958330775151,
        "score": 0.99999955454321
      },
      "images": {
        "expanded": "Base64 encoded image"
      },
      "tracking_id": "ac6g877c-e171-11e9-9194-0242ac110002",
      "velocity": 5.042970657348633,
      "velocity_state": "moving"
    }
  ],
  "vehicles": [
    {
      "bounding_box": {
        "origin": {
          "x": 176.9546213548795,
          "y": 183.1548653578951
        },
        "width": 192.17248916625977,
        "height": 92.03958330775151,
        "score": 0.89999955454321
      },
      "images": {
        "expanded": "Base64 encoded image"
      },
      "tracking_id": "b8p9d5gy-e171-11e9-9194-0242ac110002",
      "velocity": 31.042970657348633,
      "velocity_state": "moving"
    }
  ],
  "metadata": {
    "width": 1080,
    "height": 674,
    "codec": "h264",
    "frame_rate": "30/1"
  },
  "timestamp": "2019-03-29T23:16:13.647072+00:00",
  "source_timestamp": "2019-03-29T23:16:13.647072+00:00"
}
```

Receiving Alerts

Connection path	Event
`ws://<host>:5051/alerts`	`alert`

The streaming container will push alerts for important events that occur during the processing of frame data. Some of these events will need to be captured by the client for further action such as when the container is unable to suddenly read frames from the stream. To receive these events, use a Socket.IO ↗ library in the platform of your choice, connect to `ws://<host>:5051/alerts` and listen for alerts on the `alert` channel. Below are the alerts the streaming container currently supports. An alert with a `critical` level indicates some client action is required.

Event payloads are JSONs sent in string representation in event `data` field.

Type (level)	Description	Client action
decoder_stream_down (critical)	Decoder app received too many frame read errors. Stopping stream.	Retry decoding if/when appropriate.
decoder_read_frame (error)	Decoder app is unable to read frame from stream.	
cannot_start_ffmpeg (critical)	Decoder app is unable to start FFmpeg decoding process.	
shmem_allocation_failed (critical)	The container is unable to allocate shared memory.	Reduce the number of processed streams, lower the `frame_rate` for the streams being processed, or increase the `shm-size`.
frame_skipped (critical)	Inference queue is full. Decoder app is dropping frame.	Increase the number of inference workers, decrease the number of streams, or lower the `frame_rate` for the streams being processed.

new_objects_tracked (info)	New faces, persons, or vehicles have been tracked. When this alert is generated, it may contain extra information in the alert such as `stream_id`, `object_type`, `total_tracked`, and `new_tracking_ids`.	
expired_object_tracks (info)	Expired tracks for faces, persons, or vehicles. When this alert is generated, it may contain extra information in the alert such as `stream_id`, `object_type`, `total_tracked`, and `expired_tracking_ids`.	

Example

C#

```
using System;
using System.Text;
using SocketIOClient;

namespace Example
{
    class Program
    {
        private const string NAMESPACE = "/alerts";
        private const string STREAM_URL = "ws://<host>:5051";

        static void Main(string[] args)
        {
            var sio = new SocketIO(new Uri(STREAM_URL + NAMESPACE),
new SocketIOOptions
            {
                Transport =
SocketIOClient.Transport.TransportProtocol.WebSocket,
            });

            sio.OnConnected += (sender, ev) =>
            {
                Console.WriteLine("Connected");
            };

            sio.OnDisconnected += (sender, ev) =>
            {
                Console.WriteLine("Disconnected");
            };

            sio.OnError += (sender, error) =>
            {
                Console.WriteLine("Error: {0}", error);
            };

            sio.On("alert", ev =>
            {

Console.WriteLine(ev.GetValue().GetProperty("data").GetString());
            });

            sio.ConnectAsync().Wait();

            Console.ReadLine();
        }
    }
}
```

C++

```
#include <iostream>
#include <thread>

#include <sio_client.h>

using namespace std;

const char *NAMESPACE = "/alerts";
const char *STREAM_URL = "ws://<host>:5051";

int main(int argc, char *argv[]) {
  sio::client client;
  client.set_socket_open_listener(
      [&](const string &) { cout << "Connected" << endl; });
  client.set_socket_close_listener(
      [](const string &) { cout << "Disconnected" << endl; });
  client.set_fail_listener(
      []() { cout << "Failure" << endl; });

  client.connect(STREAM_URL);

  client.socket(NAMESPACE)->on_error(
      [&](sio::message::ptr const &msg) { cout << "Error: " << msg-
>get_string() << endl; });

  client.socket(NAMESPACE)->on("alert",
      [&](const string &, const sio::message::ptr &message, bool,
sio::message::list &) { cout << message->get_map().at("data")-
>get_string() << endl; });

  while (true)
    this_thread::sleep_for(1s);

  return 0;
}
```

Java

```
import io.socket.client.*;
import io.socket.emitter.Emitter;
import io.socket.engineio.client.transports.WebSocket;
import java.net.URISyntaxException;
import org.json.JSONObject;

public class Example {
    public static final String NAMESPACE = "/alerts";
    public static final String STREAM_URL = "ws://<host>:5051";

    public static void main(String[] args) throws URISyntaxException
{
        IO.Options options =
                IO.Options.builder().setTransports(new String[]
{WebSocket.NAME}).build();
        Socket sio = IO.socket(STREAM_URL + NAMESPACE, options);

        sio.on(
                Socket.EVENT_CONNECT,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        System.out.println("Connected");
                    }
                });

        sio.on(
                Socket.EVENT_DISCONNECT,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        System.out.println("Disconnected");
                    }
                });

        sio.on(
                Socket.EVENT_CONNECT_ERROR,
                new Emitter.Listener() {
                    @Override
                    public void call(Object... args) {
                        String err = (String) args[0];
                        System.out.printf("Error: %s%n", err);
                    }
                });

        sio.on(
                "alert",
                new Emitter.Listener() {
```

```
                    @Override
                    public void call(Object... args) {
                        JSONObject data = (JSONObject) args[0];
                        System.out.printf("%s%n", data);
                    }
                });

        sio.connect();

        try {
            while (true) Thread.sleep(Long.MAX_VALUE);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Node

```
const io = require("socket.io-client");

const NAMESPACE = "/alerts";
const STREAM_URL = "ws://<host>:5051";

var sio = io.connect(STREAM_URL + NAMESPACE, {
  transports: ["websocket"]
});

sio.on("connect", () => {
  console.log("Connected");
})

sio.on("disconnect", () => {
  console.log("Disconnected");
});

sio.on("connect_error", (err) => {
  console.log("Error: " + err);
});

sio.on("alert", (msg) => {
  var data = JSON.parse(msg.data);
  console.log(data);
});
```

Python

```python
import asyncio
import json

import socketio

NAMESPACE = "/alerts"
STREAM_URL = "ws://<host>:5051"

sio = socketio.AsyncClient()


@sio.on("connect")
async def connect():
    print("Connected")


@sio.on("disconnect")
async def disconnect():
    print("Disconnected")


@sio.on("connect_error")
async def connect_error(error):
    print("Error: " + error)


@sio.on("alert", namespace=NAMESPACE)
async def alert(msg):
    parsed = json.loads(msg["data"])
    print(json.dumps(parsed, indent=2))


async def main(host=STREAM_URL):
    await sio.connect(host, namespaces=[NAMESPACE], transports=
["websocket"])
    await sio.wait()


if __name__ == "__main__":
    asyncio.run(main())
```

The socket events contain JSON structured data like this:

```
{
   "level": "info",
   "type": "new_objects_tracked",
   "message": "New objects tracked.",
   "stream": "stream1",
   "timestamp": "2020-11-26T22:32:49.489293+00:00",
   "stream_id": "297986c2-d5a4-11ed-9000-0242ac110004",
   "object_type": "face",
   "total_tracked": 3,
   "new_tracking_ids": [
     "8abe3b22-f4bb-11ed-81b9-0242ac110002",
     "8abe3b86-f4bb-11ed-81b9-0242ac110002",
     "8abe3898-f4bb-11ed-81b9-0242ac110002"
   ]
}
```

Cross-stream Tracking

Streaming has the ability to track similar objects across multiple streams as the same object by giving it the same tracking ID across these streams. In order to utilize this feature the `tracking_group` Start Decoding API parameter must be set to the same value across all the relevant streams that are desired to group together, i.e., starting two separate streams with the `"tracking_group": "group1"` API parameter will enable group-tracking for these two streams.

> This ability is only applied to face tracking at the moment, person and vehicle tracking cannot take advantage of this functionality.

When the tracked object expires in all the streams that are grouped together (see `face_tracking_duration_seconds in` Start Decoding, or all the streams that are grouped together are no longer processing (via Stop Decoding), the corresponding tracked object from that group will also expire.

> This feature is limited to each individual Streaming instance (there's no communication/coordination between Streaming instances on any level at the moment).

Release notes

v8.0.2

July 10, 2024

This update exposes `source_timestamp` from the Streaming Socket.IO Outputs and addresses bugs in the Cross-stream Tracking.

Fixes

- When utilizing Cross-stream Tracking, various errors could occur depending on the timing and sequences of the face trackings and expirations across video streams configured to utilize this feature. The validation and handling of this logic has been improved and these bugs have now been addressed.
- Addressed the following Python CVEs:
  - CVE-2024-34069 | Upgraded to `werkzeug==3.0.3`
  - CVE-2024-1135 | Upgraded to `gunicorn==22.0.0`
  - CVE-2024-1681 | Upgraded to `flask-cors==4.0.1`
  - CVE-2024-22195, CVE-2024-34064 | Upgraded to `jinja2==3.1.4`
  - CVE-2024-3651 | Upgraded to `idna==3.7`
  - CVE-2024-35195 | Upgraded to `requests==2.32.0`
  - CVE-2024-37891 | Upgraded to `urllib3==2.2.2`

Features

- Introducing the ability to acquire and output the FFmpeg PRFT ↗ frame metadata if provided within the frames of the client's video stream. This can be enabled via PV_SOURCE_TIMESTAMP and if found, Streaming will output the field `source_timestamp` containing this ISO formatted timestamp via Socket.IO Outputs.

> Streaming will now always output the `source_timestamp` from its events, however if the environmental variable is not provided OR if the PRFT frame metadata is not found in the frame processed, this field will contain a `null` value.

Known Issues and Limitation

- The computed roll/pitch/yaw from Streaming may be less accurate since v8.0.0 (will need more investigation).

v8.0.1

January 29, 2024

This update exposes `attributes_images` from the Streaming Socket.IO Outputs.

Features

- Now exposing the `attributes_images` from the Streaming Socket.IO Outputs for each face detected when the PV_OUTPUT_ATTRIBUTES_IMAGES or the `output_attributes_images` API configuration in the Start Decoding Request Parameters is enabled. The `attributes_images` can be directly provided to the Paravision Processor's `process_prepared_face_image` API to compute the Attributes of the corresponding face.

> Streaming does not provide Attributes processing directly from its output, therefore it can be paired with Paravision Processor to process the Attributes for faces detected.

Known Issues and Limitation

- The Streaming container startup time with TensorRT can take (tens of) minutes even when PV_TRT_ENGINE_PATH is set and the serialized model files are

utilized, this may be a limitation of TensorRT but it is being investigated for future improvements
- With the update to the Landmarks model (Gen6), the computed roll/pitch/yaw from Streaming may be less accurate (will need more investigation)

v8.0.0

August 31, 2023

Initial Streaming v8 GA release. Supports and utilizes Paravision Gen6 Face Recognition, along with the latest Attributes, Person, and Vehicle SDKs. Also provides enhanced capabilities and features from its predecessor as outlined in the Installation and Usage guides.

Known Issues and Limitation

- The Streaming container startup time with TensorRT can take (tens of) minutes even when PV_TRT_ENGINE_PATH is set and the serialized model files are utilized, this may be a limitation of TensorRT but it is being investigated for future improvements
- With the update to the Landmarks model (Gen6), the computed roll/pitch/yaw from Streaming may be less accurate (will need more investigation)